

Part I.

```

[('9 may 1998', '1998-05-09'),
 ('10.11.19', '2019-11-10'),
 ('9/10/70', '1970-09-10'),
 ('saturday april 28 1990', '1990-04-28'),
 ('thursday january 26 1995', '1995-01-26'),
 ('monday march 7 1983', '1983-03-07'),
 ('sunday may 22 1988', '1988-05-22'),
 ('08 jul 2008', '2008-07-08'),
 ('8 sep 1999', '1999-09-08'),
 ('thursday january 1 1981', '1981-01-01')]

(dataset)

array([12,  0, 24, 13, 34,  0,  4, 12, 12, 11, 36, 36, 36, 36, 36, 36, 36,
       36, 36, 36, 36, 36, 36, 36, 36, 36, 36, 36, 36])

(X)

array([ 2, 10, 10,  9,  0,  1,  6,  0,  1, 10])

(Y)

{'-': 0,
 '0': 1,
 '1': 2,
 '2': 3,
 '3': 4,
 '4': 5,
 '5': 6,
 '6': 7,
 '7': 8,
 '8': 9,
 '9': 10}

(machine_vocab)

{'0': '-',
 '1': '0',
 '2': '1',
 '3': '2',
 '4': '3',
 '5': '4',
 '6': '5',
 '7': '6',
 '8': '7',
 '9': '8',
 '10': '9'}

(inv_machine_vocab)

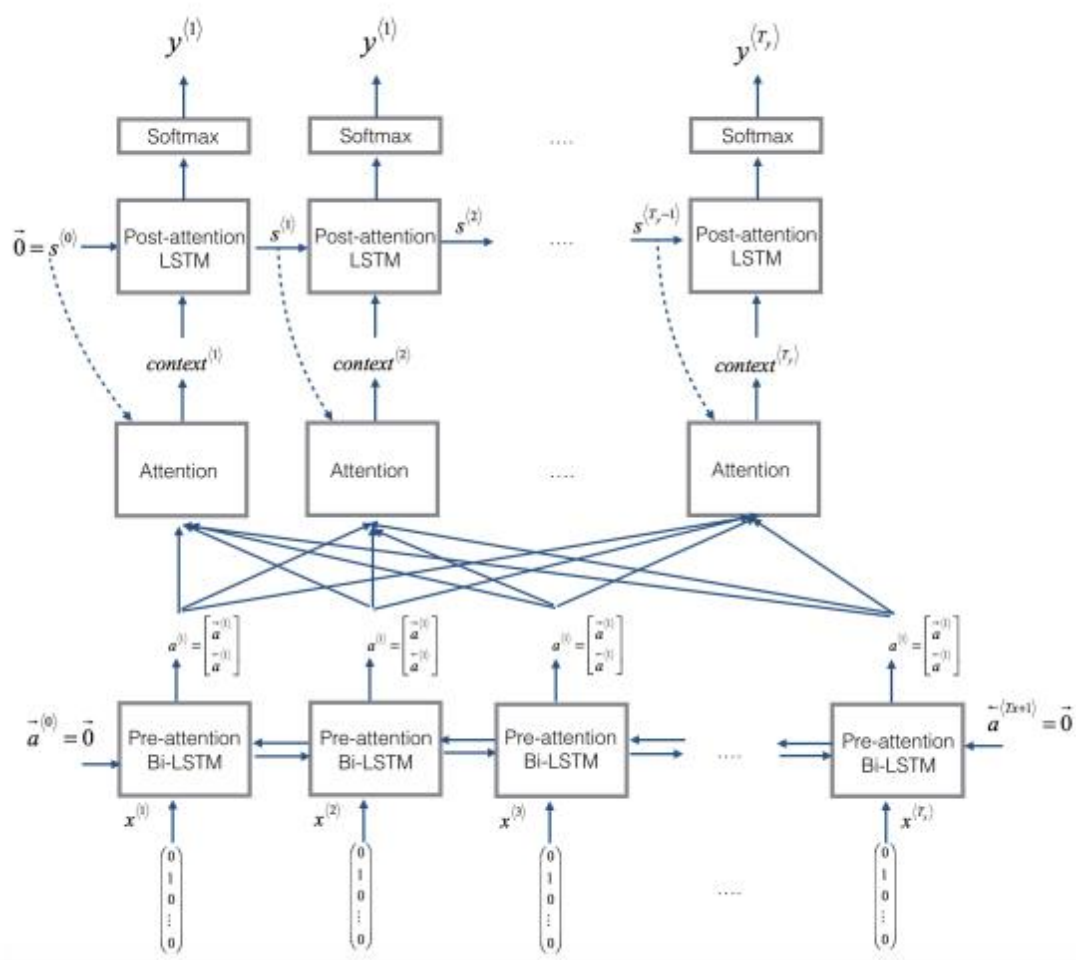
{' ': 0,
 '.': 1,
 '/': 2,
 '0': 3,
 '1': 4,
 '2': 5,
 '3': 6,
 '4': 7,
 '5': 8,
 '6': 9,
 '7': 10,
 '8': 11,
 '9': 12,
 'a': 13,
 'b': 14,
 'c': 15,
 'd': 16,
 'e': 17,
 'f': 18,
 'g': 19,
 'h': 20,
 'i': 21,
 'j': 22,
 'l': 23,
 'm': 24,
 'n': 25,
 'o': 26,
 'p': 27,
 'r': 28,
 's': 29,
 't': 30,
 'u': 31,
 'v': 32,
 'w': 33,
 'y': 34,
 '<unk>': 35,
 '<pad>': 36}

```

觀察資料型態，將 dataset 與 X 打映出來，可以發現 X 是將 dataset 的資料經由 human_vocab 轉換成 vector，例如 9 對應 12、m 對應 24，以此類推。

而 Y 則是透過 machine_vocab 轉換成 vector，例如 1998 對應 2、10、10、9

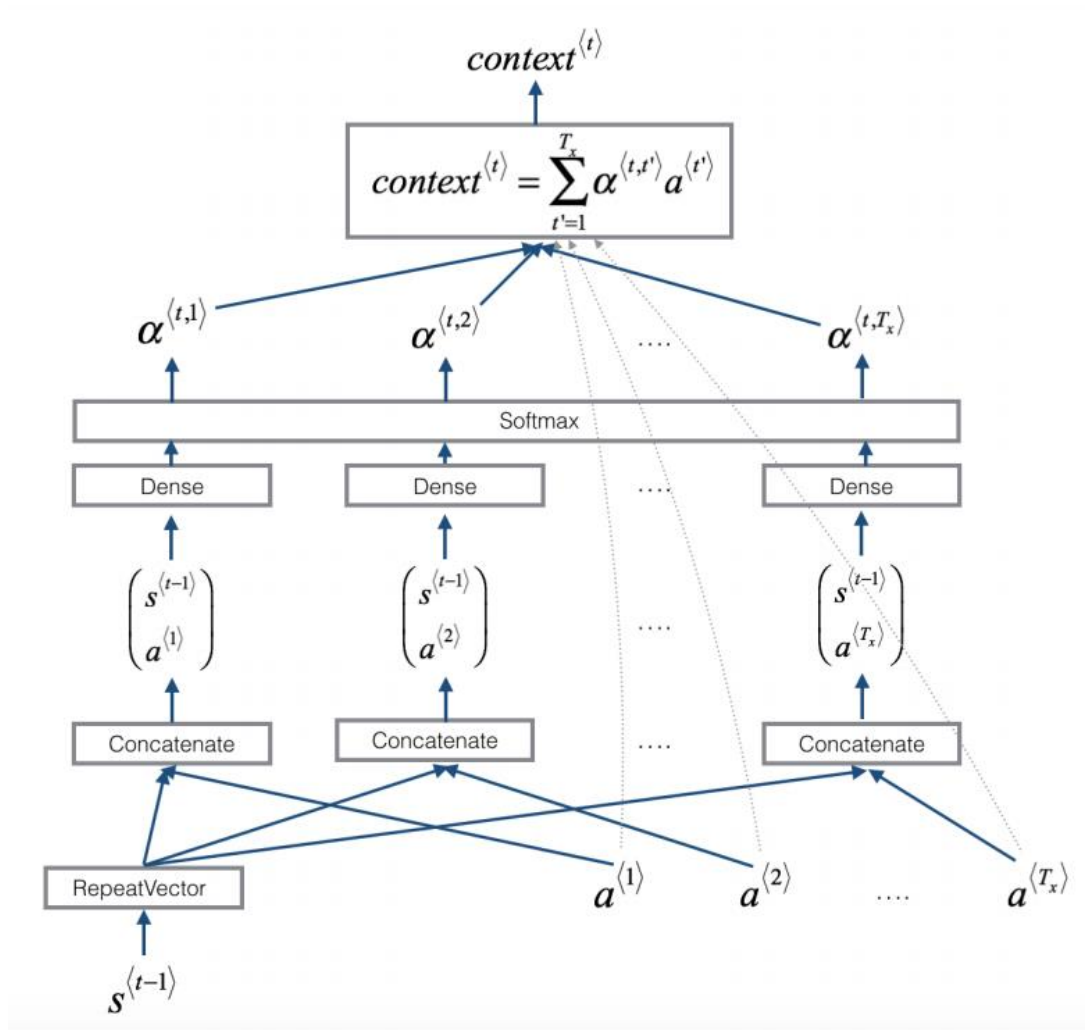
由於資料量很少，可以使用 one hot。



(模型架構)

首先能看到第一層的 Bi-LSTM 在 attention 之前，因此稱 pre-attention，Bi-LSTM 中文譯作雙向 LSTM，與一般的 LSTM 不同的地方在於它會由左至右做一次 LSTM 與由右至左做一次 LSTM 總共做兩次，即雙向的意思，這能改善 LSTM 推進時，後者重要性大於前者的問題。前幾次的作業所做內容是編寫句子，編寫句子只需要注意後方詞彙與前者詞彙關係，第一個字出來就可以預測第二個字是甚麼，例如 I 可以預測到下一個字可能是 am，但本次欲實作日期轉換，我們的日期並非只有單一的前後關係，由於順序不固定，因此前後關係必須考慮到雙向，例如 May 5 2019，換成機器語言是 2019-05-05，很明顯年分在日期後方或者前方都具有意義，我們必須依靠雙向 LSTM 訓練才有較佳的效果^[1]。

第二個 LSTM 在 attention 之後，因此稱做 post-attention，這個 LSTM 和前幾次的作業不同，並沒有將輸出(y)當作下一個 LSTM 的輸入，這是因為字詞之間的關聯性並不高，不像 I am 這種強烈關係，只是日期、年分，兩者並沒有一定的關係，因此我們不必將輸出當作下一個 LSTM 的輸入。



```
repeater = RepeatVector(Tx)
concatenator = Concatenate(axis=-1)
densor1 = Dense(10, activation = "tanh")
densor2 = Dense(1, activation = "relu")
activator = Activation(softmax, name='attention_weights')
dotor = Dot(axes = 1)
```

```
### START CODE HERE ###
# Use repeator to repeat s_prev to
s_prev = repeator(s_prev)
# Use concatenator to concatenate
concat = concatenator([a,s_prev])
# Use densor1 to propagate concat
e = densor1(concat)
# Use densor2 to propagate e throu
energies = densor2(e)
# Use "activator" on "energies" to
alphas = activator(energies)
# Use dotor together with "alphas"
context = dotor([alphas,a])
### END CODE HERE ###
```

在 attention 中，我們使用 `RepeatVector` 將 S 複製 T_x 個送入 `Concatenate`，而這邊和結構圖不同的地方在於有兩層 `Dense`，第一層 `Dense` 輸出為 10、激活使用 `tanh` 限制輸出於 $[-1,1]$ ，第二層輸出為 1 激活使用 `relu`，再經過 `softmax` 做機率處理，計算詞彙的前後關係。

```
Epoch 10/10
10000/10000 [=====] - 8s 780us/step - loss: 0.0284 - dense_3_loss: 6.9345e-04 - dense_3_acc: 1.0000
- dense_3_acc_1: 1.0000 - dense_3_acc_2: 0.9998 - dense_3_acc_3: 0.9979 - dense_3_acc_4: 1.0000 - dense_3_acc_5: 1.0000 - den
se_3_acc_6: 1.0000 - dense_3_acc_7: 1.0000 - dense_3_acc_8: 1.0000 - dense_3_acc_9: 1.0000
```

```
source: 3 May 1979
output: 2018-07-21
source: 5 April 09
output: 2083-07-04
source: 21th of August 2016
output: 2016-18-18
source: Tue 10 Jul 2007
output: 2033-07-08
source: Saturday May 9 2018
output: 2014-10-19
source: March 3 2001
output: 1983-03-16
source: March 3rd 2001
output: 1983-09-16
source: 1 March 2001
output: 1988-07-17
```

將 model run 10 epochs 的結果可以發現正確率全部都是 1，但是丟入未見過的日期進去，成效不彰，overfitting 了。

```
Epoch 10/10
10000/10000 [=====] - 96s 10ms/step - loss: 1.8420 - dense_3_loss: 0.3913 - dense_3_acc: 0.9595 - de
nse_3_acc_1: 0.9673 - dense_3_acc_2: 0.9170 - dense_3_acc_3: 0.9481 - dense_3_acc_4: 1.0000 - dense_3_acc_5: 0.9703 - dense_3
_acc_6: 0.9084 - dense_3_acc_7: 1.0000 - dense_3_acc_8: 0.8730 - dense_3_acc_9: 0.8742
```

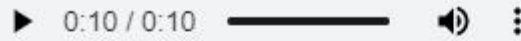
```
source: 3 May 1979
output: 1983-01-22
source: 5 April 09
output: 1914-03-04
source: 21th of August 2016
output: 1985-01-14
source: Tue 10 Jul 2007
output: 1013-03-03
source: Saturday May 9 2018
output: 1911-10-09
source: March 3 2001
output: 1974-01-24
source: March 3rd 2001
output: 1984-08-26
source: 1 March 2001
output: 1974-08-08
```

修改 batch size=4，以及 beta 值發現雖然正確率不再全部都是 1 但是對於未見過的資料仍然無法正確轉換，評估訓練語言的模型並不能直接由正確率判斷模型的好壞，丟入未見過的資料讓模型預測是最簡單檢查模型的方法，與 CNN 不同的是，無法靠著修改參數就能有良好的訓練成果，必須從結構或其他方面下手。

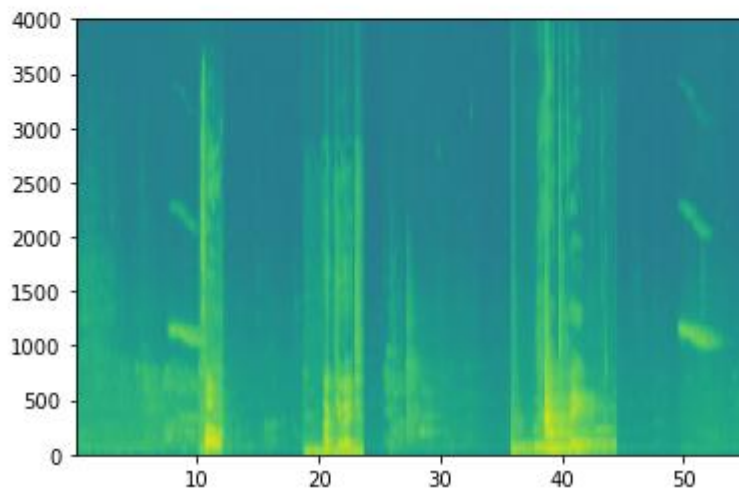
Part II.

```
In [6]: 1 IPython.display.Audio("audio_examples/example_train.wav")
```

Out[6]:



```
In [7]: 1 x = graph_spectrogram("audio_examples/example_train.wav")
```



通常語音辨識會從頻譜下手，觀察頻譜可以發現在 $x=10$ 、 20 、 40 的位置有出現不同頻率的聲音，比對語音檔可以聽到在 1 秒、3 秒、7 秒的地方有人聲。進一步分析前兩個語音是男聲，通常會較低頻，比對頻譜也確實在低頻部分顯著許多，最後一個語音則是女聲，跨越的頻率較多，高頻也較明顯。

- 441000 (raw audio)
- $5511 = T_x$ (spectrogram output, and dimension of input to the neural network).
- 10000 (used by the 'pydub' module to synthesize audio)
- $1375 = T_y$ (the number of steps in the output of the GRU you'll build).

這些參數除了 441000 是人聲取樣頻率的公認值，其於參數是可以自己調整，而總長度都是 10s。例如 T_y 將 10s 切成 1375 等份，每等份表 0.0072s，這是用來處理是否偵測到 activate 的頻率。10000 則是因為 Pydub 使用 1ms 當作切割區間，我們的音檔長度為 10s，所以共需要 10000 個 timesteps。

```

1 activates, negatives, backgrounds = load_raw_audio()
2
3 print("background len: " + str(len(backgrounds[0])))
4 print("activate[0] len: " + str(len(activates[0])))
5 print("activate[1] len: " + str(len(activates[1])))

```

background len: 10000
activate[0] len: 721
activate[1] len: 731

我們將隨便一個具有 activate 的音檔映出長度，可以知道發一個 activate 的音大概是 0.72 秒左右。

```

def insert_ones(y, segment_end_ms):
    """
    Update the label vector y. The labels of the 50 output steps should be set to 1. By strictly we mean that the label of segment_end_ms + 50 following labels should be ones.

    Arguments:
    y -- numpy array of shape (1, Ty), the labels of the training
    segment_end_ms -- the end time of the segment in ms

    Returns:
    y -- updated labels
    """

    # duration of the background (in terms of spectrogram time-steps)
    segment_end_y = int(segment_end_ms * Ty / 10000.0)

    # Add 1 to the correct index in the background label (y)
    ### START CODE HERE ### (~ 3 lines)
    for i in range(segment_end_y+1, segment_end_y+51):
        if i < Ty:
            y[0, i] = 1.0
    ### END CODE HERE ###

    return y

```

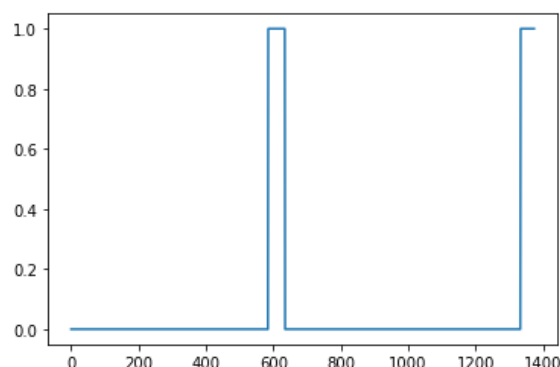
我們在標注 activate 是取發音結束時再多 50 個 timesteps，50 個 timesteps 換算成時間為 0.36 秒，加上發音時間總長約 1 秒，會取 50 是因為當人聽到發音時，加注上去需要的反應時間大概就是這個時長。function 中的 $segment_end_ms * Ty / 10000.0$ 用來轉換，將以 ms 為單位的 segment_end_ms 轉換成 Ty 的對應間隔。例如第 5 秒結束發音，segment_end_ms=5000，換算成 Ty 應發生於 687。

```

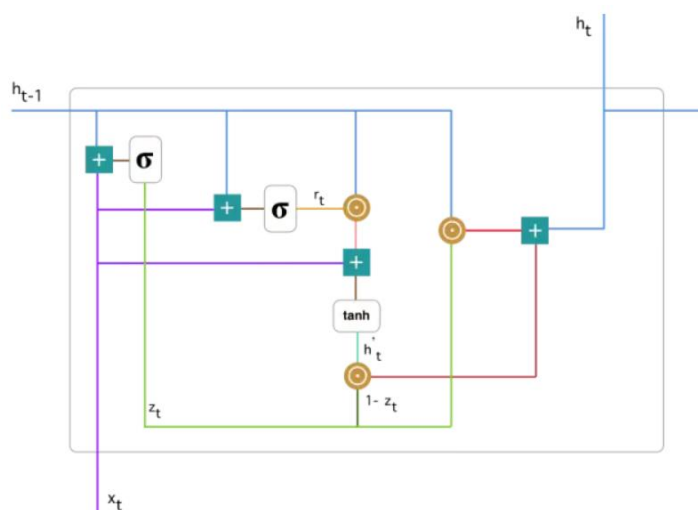
1 arr1 = insert_ones(np.zeros((1, Ty)), 9700)
2 plt.plot(insert_ones(arr1, 4251)[0,:])
3 print("sanity checks:", arr1[0][1333], arr1[0][634], arr1[0][635])

```

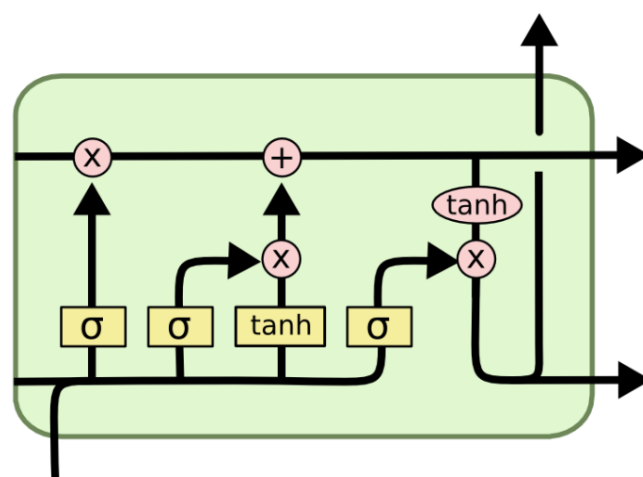
sanity checks: 0.0 1.0 0.0



觀察範例，我們要在 9.7 秒末結束，換算成 T_y 格式為第 1333.75 位，因此 1334~1375 位為 1，4.251 秒末結束，換算成 T_y 為 584.5 位，因此 585~634 位為 1。必須注意的點是插入的地方不能重複插入，所以使用 "is_overlapping()" 判斷是否重複。我們可以依照此流程製作自己的語音：隨機選取背景音->插入 activate 或其他字句->檢查是否重複插入->修改 y label->完成訓練集。

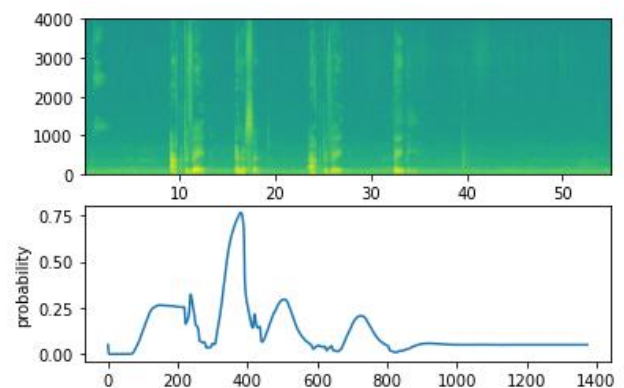
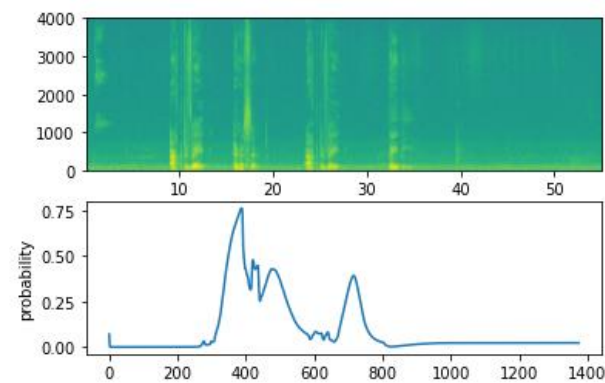
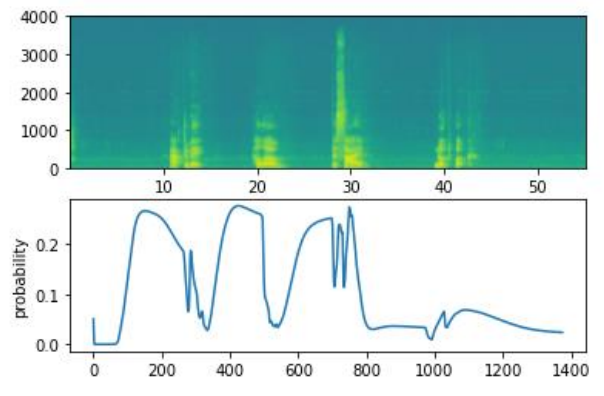
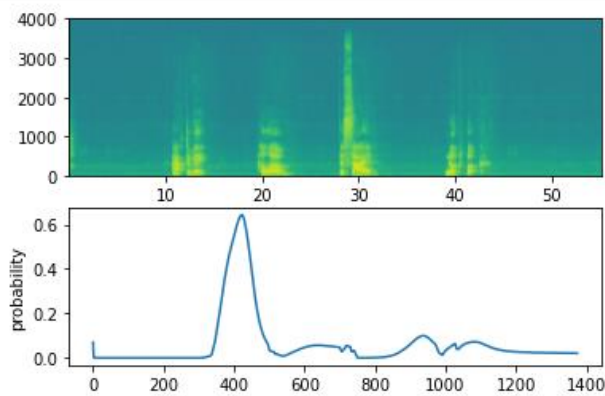


(GRU)

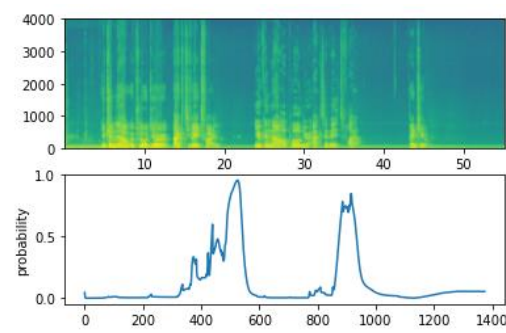
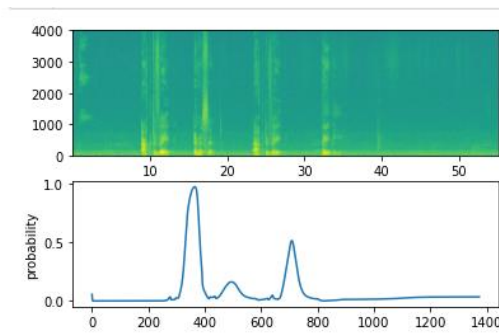
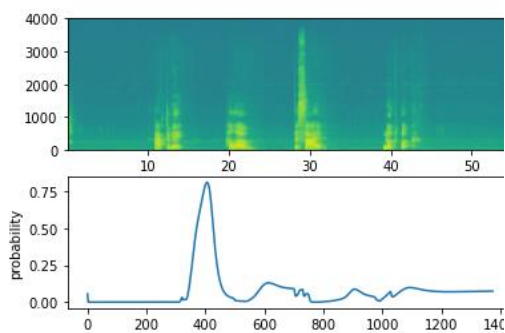


(LSTM)

GRU 將 LSTM 中的遺忘閥 (forget gate) 與輸入閥 (input gate) 用一個更新閥 (update gate) 取代，並把單元狀態 (cell state) 和隱藏狀態 (h_t) 進行合併，因此 GRU 簡化了 LSTM，運算加快，對於資料少可能無感，但在資料量大時，GRU 耗時短、防止過擬合使之更受歡迎^{[2][3]}。



經由上面的圖比對可以發現，重新載入 **model** 再跑一次出來的預測值並不同，甚至會預測錯誤，每一次的權重不同所以導致某些錯判，可以看到右上圖基本上是只要有人聲就會判斷 **True**，右下的圖也只判斷出一個 **activate**。



載入多次後找到了一個預測比較好的結果，對於每一個 **activate** 都能正確指認出並加上鐘聲，其參數如下

```
Epoch 1/1
26/26 [=====] - 13s 483ms/step - loss: 0.0638 - acc: 0.9796
```

```
<keras.callbacks.History at 0x1ed859e2f60>
```

```
1 loss, acc = model.evaluate(X_dev, Y_dev)
2 print("Dev set accuracy = ", acc)
```

```
25/25 [=====] - 2s 68ms/step
Dev set accuracy = 0.9338472485542297
```


reference :

[1]

<https://www.itread01.com/content/1546620066.html>

[2]

<https://medium.com/@tengyuanchang/%E6%AF%94%E8%BC%83%E9%95%B7%E7%9F%AD%E6%9C%9F%E8%A8%98%E6%86%B6%E6%A8%A1%E5%9E%8B-lstm-%E8%88%87%E6%94%B9%E8%89%AF%E5%BE%8C%E7%9A%84%E9%81%9E%E6%A D%B8%E7%A5%9E%E7%B6%93%E7%B6%B2%E8%B7%AF%E6%A8%A1%E5%9E%8B-gru-813dec22ec6d>

[3]

<https://blog.csdn.net/fendouaini/article/details/80272320>